

Relational-Indexed Retrieval-Augmented Generation (RI-RAG)

Laurie Dieudonné MAVOUNGOU

JK AI

lmavoungou@outlook.be

ABSTRACT

Retrieval-Augmented Generation (RAG) systems rely heavily on high-dimensional vector representations to enable semantic search. However, vector-based retrieval architectures may store substantial payload information and metadata alongside vector representations, motivating investigation into alternative storage organizations. This paper introduces *Relational-Indexed Retrieval-Augmented Generation* (RI-RAG), a hybrid architecture that integrates relational database management systems (RDBMS) with vector stores to separate semantic indexing from authoritative relational storage. RI-RAG maps lightweight semantic representations to structured relational records through record identifiers, allowing the vector layer to serve as the semantic retrieval entry point while the relational database remains the authoritative source for complete content and structured metadata. The prototype was evaluated through three experimental rounds. The first compared an initial Standard RAG configuration with RI-RAG. The second refined the vector-store representation measurement and repeated retrieval-latency evaluation, obtaining a measured 24.70% reduction in the materialized vector-store representation for the evaluated 13-record corpus. The third evaluated retrieval correspondence, evidence grounding, and the configured database-access boundary. Across ten ground-truth queries, RI-RAG achieved 90% top-1 retrieval accuracy, with a precision of 0.9000 and an F1 score of 0.9474. The results represent prototype measurements under controlled conditions rather than evidence of large-scale enterprise scalability.

KEYWORDS

Relational database management systems, Information Retrieval, RAG

1 OVERVIEW

Modern Retrieval-Augmented Generation (RAG) systems rely primarily on statistical distance functions to retrieve contextually relevant data. However, semantic similarity retrieval alone does not provide an explicit relational source of truth for the retrieved record. To address this limitation, we present a hybrid retrieval architecture in which candidate vector references are mapped to corresponding records in a relational database. This additional retrieval stage separates semantic candidate selection from authoritative record retrieval.

By isolating full unstructured payloads within the relational layer and maintaining compact semantic representations and record identifiers in the vector layer, RI-RAG provides an alternative storage organization whose resource behavior can be measured experimentally. The architecture is intended to reduce the amount of payload information materialized within the vector-store layer rather than to establish a universal reduction in total system memory consumption. The initial benchmarks therefore measure vector-store

representation size and retrieval latency for the implemented prototype.

The relational layer can also provide a boundary at which database access-control and integrity mechanisms may be applied. These properties depend on the database and application configuration; the experiments in this report evaluate only the configured database-access behavior rather than comprehensive enterprise security.

Section 2 discusses the related work, and Section 3 describes the research problem in detail, along with the proposed solution. Section 4 presents the dataset used and the prototyping scope. The empirical benchmarks and performance analysis are presented in Section 5. Section 6 presents some AI foundational concepts and how RI-RAG intends to address them. Section 7 presents the broader impact of RI-RAG and some target applications, while Section 8 refers to the GitHub repository containing the source code for this paper. The conclusion and future work are presented in Section 9, while the AI usage acknowledgement is presented in Section 10.

2 RELATED WORK

Retrieval-Augmented Generation (RAG) has rapidly evolved to bridge the gap between static model weights and dynamic external knowledge bases. Existing implementations can involve challenges related to memory organization, structural grounding, and access control. In this section, we discuss prior approaches across these dimensions and describe the design distinction of *Relational-Indexed Retrieval-Augmented Generation* (RI-RAG).

2.1 Memory Optimization and Hardware Offloading

As context windows expand, inference costs can increase due to high-dimensional memory consumption. Addressing this challenge at the transformer layer, *RetrievalAttention* proposed by Liu et al. [3] offloads key-value (KV) caches from GPU to CPU memory, relying on dynamic vector search to recover approximate nearest neighbors during generation.

While Liu et al. [3] and our work address memory organization at different levels of the system, they operate at fundamentally different architectural tiers. *RetrievalAttention* operates on internal model activations and memory tiers, whereas RI-RAG operates at the application and database layers. RI-RAG separates full text payloads from vector-search information by storing complete authoritative records in a relational database while maintaining semantic representations and record identifiers in the vector store. This changes the organization of information retrieval and storage at the application/database layer without modifying the model's attention mechanisms or internal architecture.

2.2 Hierarchical and Structured Retrieval Paradigms

To provide more structured retrieval, recent literature has explored graph-based and hierarchical indexing approaches. Edge et al. [1] introduced GraphRAG, which extracts entity-relationship graphs to enable hierarchical community summaries. Similarly, Gutierrez et al. [2] proposed HippoRAG, a neurobiologically inspired long-term memory framework that models associative retrieval through personalized PageRank over knowledge graphs.

In contrast to graph-based approaches, RI-RAG uses relational database management systems (RDBMS) to connect semantic retrieval results with structured authoritative records through record identifiers and relational schemas. The architecture therefore provides a relational resolution step without requiring graph extraction or graph traversal.

2.3 Enterprise Security and Access Boundary Constraints

Vector stores and graph-based retrieval systems can use different approaches to access control and filtering. RI-RAG instead places the authoritative record in a relational layer, creating a database boundary at which access-control policies can potentially be applied.

RI-RAG decouples the vector representation from the underlying enterprise record. By routing retrieved candidates through a relational layer, the architecture provides a point at which database-level privileges and application-level access-control policies can be applied before relational content is returned to the generation component. The effectiveness of such controls depends on the deployment configuration.

3 RESEARCH PROBLEM

Existing RAG implementations present three architectural considerations addressed by this work:

- (1) Semantic retrieval alone does not necessarily expose an explicit relationship between a retrieved vector representation and an authoritative structured record.
- (2) Vector indexes may contain payloads and metadata in addition to embeddings. Separating full payload storage from semantic indexing provides an alternative storage organization whose resource behavior can be measured experimentally.
- (3) Enterprise deployments require access-control and audit mechanisms around retrieved information. RI-RAG places authoritative records in a relational database where such mechanisms can be configured, although the present report does not provide a comprehensive security evaluation.

3.1 Contributions

The contributions of this work are:

- Introduction of the RI-RAG hybrid relational-vector retrieval architecture.
- Prototype implementation using semantic summaries in a vector store and authoritative records in MySQL.
- Experimental comparison of the materialized vector-store representation between Standard RAG and RI-RAG.

- Evaluation of retrieval accuracy using ground-truth query-to-record mappings.
- Evaluation of evidence grounding through primary-key resolution and authoritative record hydration.
- Evaluation of the configured database-access boundary.
- Identification of limitations and future experiments required for larger-scale evaluation.

3.2 Technical Methodology

RI-RAG separates semantic indexing from full document storage by using two complementary storage layers. The vector database acts as the semantic entry point for retrieval, while the relational database acts as the authoritative source of truth for complete document content and structured metadata.

The semantic summary of each document is used to generate its vector embedding. Instead of storing the complete document payload within the vector index, RI-RAG maintains a relationship between the vector representation and the corresponding relational record through its record identifier. The vector store therefore provides semantic retrieval, while the relational database provides record hydration and authoritative information.

The major architectural components are:

- **Relational verification layer:** Maintains the authoritative document content and structured metadata.
- **Structured record resolution:** Uses the retrieved record identifier to identify the corresponding relational record.
- **Payload separation:** Keeps full document payloads in the relational database while limiting the vector layer to semantic representations and required identifiers.

The retrieval workflow consists of the following stages:

- (1) **Semantic entry point:** The user query is converted into an embedding and compared with the vector representations stored in the vector database.
- (2) **Relational hydration:** The record identifier returned by vector retrieval is used to retrieve the corresponding authoritative record from MySQL.
- (3) **Entry-only storage:** The vector layer maintains the semantic representation and record identifier rather than the complete authoritative payload.
- (4) **Ingestion and synchronization:** The system maintains the relationship between vector entries and relational records during ingestion and updates.
- (5) **Hydration and ranking engine:** Retrieved identifiers are resolved into relational records for subsequent processing.
- (6) **Access control and filtering:** Relational database controls and application-level filtering can be applied to authoritative records after retrieval.

The RI-RAG retrieval workflow can be summarized as presented in Figure 1.

3.3 Vector Database Sample

The vector database contains the lightweight semantic representation and the relationship needed to retrieve the authoritative record. The vector database does not store the complete authoritative document content.

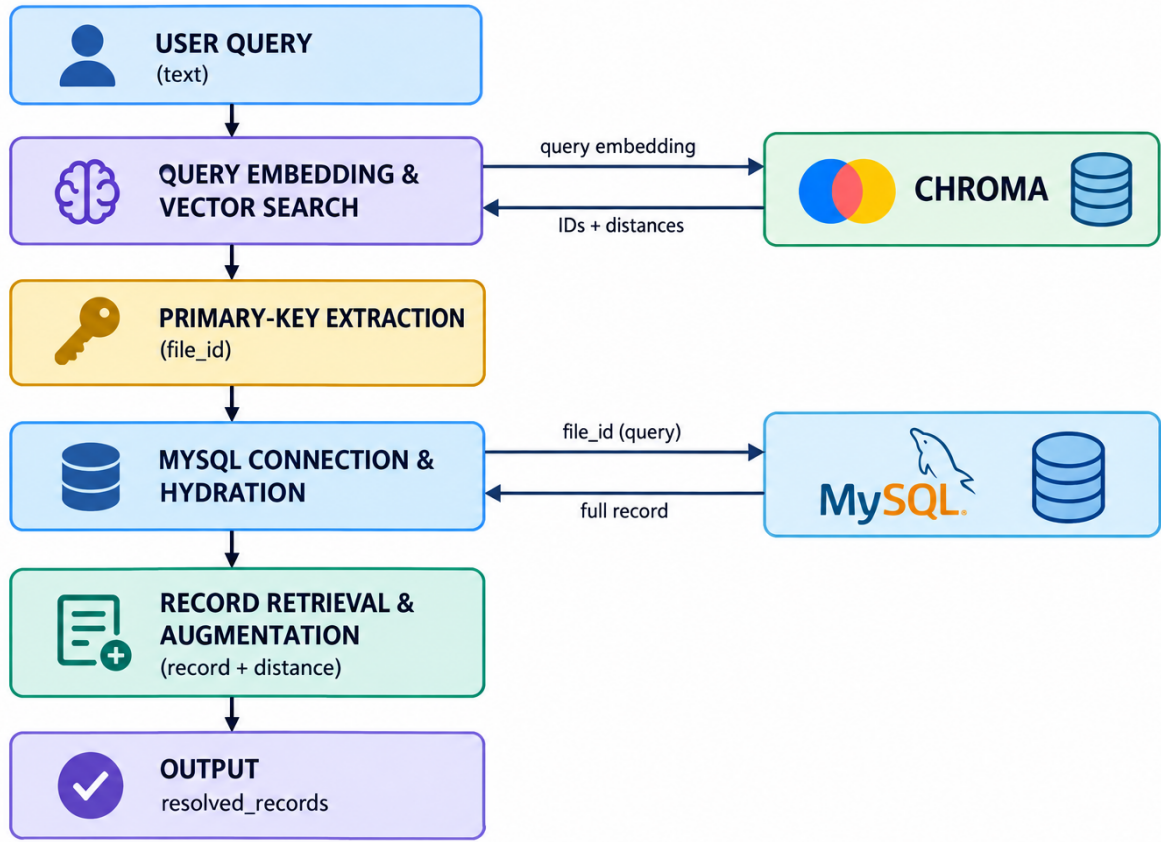


Figure 1: RI-RAG retrieval workflow.

```

{
  "id": <primary_key>,
  "metadata": {
    "file_name": <file_name>,
    "file_id": <primary_key>
  },
  "documents": <semantic_summary>,
  "embedding": <embedding>
}

```

3.4 MySQL Table Schema

MySQL serves as the authoritative source of truth for the complete document payload and structured metadata.

```

CREATE TABLE file (
  file_id VARCHAR(50) NOT NULL,
  file_name VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  semantic_summary TEXT NOT NULL,
  grade FLOAT NOT NULL,
  comments TEXT NOT NULL,

```

```

  CONSTRAINT file_pk PRIMARY KEY (file_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

```

The full content remains available in the relational layer and can be retrieved through the relationship established between the vector record identifier and the corresponding relational record.

3.5 Experimental Description

Table 1 summarizes the experimental rounds conducted in this study, their objectives, and the corresponding implementation.

The experiments intentionally distinguish between vector-store representation size, process-level memory consumption, and retrieval latency. The vector-store measurement represents the size of the Python objects materialized from the Chroma collection by the benchmark. It is therefore not interpreted as total system RAM consumption.

4 DATASET AND PROTOTYPING SCOPE

To validate the RI-RAG architectural prototype under controlled conditions, we constructed a synthetic benchmark dataset comprising 13 enterprise and security-focused records (DOC-001 through

Table 1: RI-RAG experimental evaluation.

Experiment	Objective	Implementation and Measurement
Round 1: Initial Benchmark	Compare Standard RAG and RI-RAG at the vector-store and retrieval levels.	Both systems were populated from the same MySQL corpus. The experiment measured the materialized representation returned by the Chroma collection and externally measured retrieval latency. The initial measurement produced 8,872 B for Standard RAG and 8,273 B for RI-RAG, with corresponding mean retrieval times of 7.31 ms and 22.49 ms.
Round 2: Refined Storage and Performance Evaluation	Measure the vector-store representation under the refined benchmark implementation and characterize repeated retrieval latency.	A recursive deep-size measurement was applied to the materialized Chroma collection returned by <code>collection.get()</code> . The benchmark stored full content in Standard RAG and only the semantic summary, record identifier, and file name in RI-RAG. The refined experiment measured 63,954 B for Standard RAG and 48,160 B for RI-RAG. Ten repeated retrieval measurements were also used to characterize latency.
Round 3: Retrieval and Functional Evaluation	Evaluate retrieval correspondence, evidence grounding, and the configured database-access boundary.	Ten ground-truth queries were executed through the live HybridRIRAGSystem using top-1 retrieval. The expected record identifiers were compared with the retrieved identifiers. Evidence grounding was tested using DOC-001 and authoritative MySQL hydration. Database-boundary behavior was tested using the configured <code>test_user</code> account and <code>CURRENT_USER()</code> .

DOC-013). The dataset is intentionally small and serves to demonstrate the relational-vector decoupling and retrieval workflow rather than to establish large-scale production behavior.

The dataset is modeled within a relational schema enforced via MySQL:

- **Controlled Payload Expansion:** The text payloads (content) were expanded through controlled repetition to create a reproducible test environment for examining storage behavior. This provides a controlled payload-expansion experiment rather than a representation of naturally occurring enterprise documents.
- **Domain Relevance:** The records span enterprise security and architecture topics, including consistency guidelines, autonomous agent safety, vector optimization, role-based access control, audit logging, adversarial robustness, and federated privacy.
- **Structured Attributes:** Each record contains a primary key (`file_id`), file name, full text content, semantic summary, a performance grade (FLOAT), and operational comments.

To implement and evaluate the RI-RAG prototype, specific system configurations and parameters were established across both the relational and vector storage layers:

- **Dataset Size:** The evaluation corpus consists of 13 synthetic enterprise and security-focused records (DOC-001 through DOC-013) stored in a MySQL relational database.
- **Payload Scale:** Text payloads (content) were expanded through controlled repetition, averaging approximately 750 to 1,200 characters per document while vector-side semantic summaries remained substantially shorter.
- **Embedding Model:** Semantic representations are generated using the `sentence-transformers/all-MiniLM-L6-v2` model.
- **Embedding Dimension:** The vector representation uses an embedding dimension of 384.

- **Indexing Strategy:** RI-RAG generates embeddings exclusively from the relational `semantic_summary` attribute. The complete document content remains isolated within MySQL and is retrieved through primary-key resolution.
- **Chroma Configuration:** The vector retrieval layer utilizes a local `chromadb.Client()` instance. RI-RAG stores the semantic summary as the Chroma document together with lightweight metadata containing `file_id` and `file_name`.

For the Standard RAG comparison, the vector store contains the complete content payload together with additional metadata. Consequently, the two configurations differ primarily in where the authoritative payload is stored.

Due to the limited corpus size and controlled payload construction, this initial dataset is not sufficient to establish large-scale enterprise scaling behavior. Larger corpora, more diverse documents, and substantially larger payload sizes remain necessary for future evaluation.

5 EMPIRICAL BENCHMARKS & PERFORMANCE ANALYSIS

5.1 Round 1: Initial Benchmark

The first evaluation compares a Standard RAG configuration with the RI-RAG prototype using vector-store representation size and retrieval latency. The purpose of this benchmark is to measure the effect of separating semantic indexing from authoritative relational storage.

Table 3: Round 2 refined vector-store and retrieval results.

Metric	Standard RAG	RI-RAG
Materialized representation	63,954 B	48,160 B
Representation reduction	–	24.70%
Mean retrieval latency	7.11 ms	17.75 ms
Minimum latency	5.44 ms	14.21 ms
Maximum latency	9.54 ms	25.63 ms

Table 2: Initial comparison between Standard RAG and RI-RAG.

Metric	Standard RAG	RI-RAG	Difference
Vector-store representation	8,872 B	8,273 B	599 B
Representation reduction	–	–	6.8%
Query latency	7.31 ms	22.49 ms	15.18 ms

The initial benchmark measured a 599-byte difference between the two vector-store configurations, corresponding to a 6.8% reduction in the measured materialized vector-store representation for RI-RAG. This measurement applies only to the vector-store representation returned by the benchmark and does not represent total application or system memory consumption.

The latency results show the trade-off introduced by the additional relational hydration step. Standard RAG achieved an average retrieval latency of approximately 7.31 ms, while RI-RAG measured approximately 22.49 ms under the implemented test conditions. The difference is consistent with the additional database lookup required to resolve the retrieved record identifier into authoritative content.

5.2 Round 2: Refined Storage and Retrieval Performance

The second evaluation refined the measurement procedure and used the same 13-record corpus to compare the materialized vector-store representations more directly.

The refined benchmark measured 63,954 bytes for the Standard RAG vector-store representation and 48,160 bytes for RI-RAG. The difference corresponds to a measured reduction of 24.70%:

$$\frac{63,954 - 48,160}{63,954} \times 100 \approx 24.70\%.$$

This result should be interpreted specifically as a reduction in the measured materialized vector-store representation. It should not be interpreted as a 24.70% reduction in total process RAM, because the Python runtime, embedding model, Chroma implementation, and other system components contribute additional memory independently of the stored payload representation.

The retrieval benchmark measured an average latency of 7.11 ms for Standard RAG and 17.75 ms for RI-RAG. RI-RAG therefore incurred additional latency under the evaluated conditions. This is consistent with the additional relational hydration operation required after vector retrieval.

Table 4: Round 3 functional evaluation results.

Metric	RI-RAG Result
Ground-truth queries	10
Correct top-1 retrievals	9 / 10
Top-1 retrieval accuracy	90.0%
Precision	0.9000
F1 score	0.9474
Known mismatch	DOC-003 → DOC-009
Evidence grounding	PASS
Database boundary	Verified under test_user

The Round 2 result provides an empirical observation of the intended storage trade-off: moving the complete authoritative payload out of the vector layer reduces the measured vector-store representation, while the additional relational lookup introduces retrieval overhead.

5.3 Round 3: Retrieval Accuracy, Evidence Grounding, and Database Boundary

The third evaluation examined whether the RI-RAG retrieval pipeline could resolve ground-truth records, hydrate authoritative content, and operate through the configured database-access boundary.

Ten queries were constructed with explicit expected document identifiers ranging from DOC-001 to DOC-010. Each query was submitted to the actual HybridRIRAGSystem.retrieve() implementation using top-1 retrieval.

The retrieval evaluation produced nine correct top-1 matches out of ten queries, corresponding to 90.0% top-1 retrieval accuracy. The resulting precision was 0.9000 and the F1 score was 0.9474 under the implemented evaluation formulation.

The only mismatch occurred for the query targeting DOC-003. The expected record was DOC-003, which describes the decoupling of vector search from authoritative relational records, while the system retrieved DOC-009, which concerns latency trade-offs associated with relational database round trips. The result indicates semantic overlap in the embedding space between these technically related concepts. It also illustrates that the relational architecture does not by itself guarantee semantic retrieval correctness; retrieval quality remains dependent on the semantic representation used for vector search.

For the evidence-grounding experiment, a query targeting DOC-001 was submitted through the RI-RAG retrieval pipeline. The returned record identifier was resolved through the database and the authoritative content field was inspected. The expected core sentence was present in the retrieved authoritative record, resulting in a successful grounding test. This experiment demonstrates the implemented primary-key resolution and relational hydration path rather than a comprehensive evaluation of contradiction detection across multiple generated responses.

The database-boundary experiment connected to MySQL using the configured test_user account and inspected CURRENT_USER(). The experiment returned test_user@localhost, verifying that the evaluated database operation was executed under the configured account. This demonstrates the configured database-access boundary for the tested operation. It does not establish comprehensive

enterprise RBAC effectiveness, privilege-escalation resistance, or a complete security evaluation.

6 FOUNDATIONAL CONCEPTS & PROBLEM SOLVING

6.1 What is Memory and Why Does It Matter?

In computer science and software architecture, memory refers to the high-speed volatile storage space used by applications to hold active data during processing. Memory is a finite resource, and excessive memory requirements can increase infrastructure requirements and operational costs as AI systems scale. In RAG systems, the amount of data maintained by the vector retrieval layer is therefore an important resource consideration. RI-RAG investigates whether separating semantic representations from authoritative payloads can reduce the memory and materialized data representation maintained by the vector layer.

6.2 What is Traceability and Why Does It Matter?

Traceability is the ability to associate retrieved information with its source record and relevant metadata. In retrieval systems, this association can help engineers inspect the provenance of information supplied to the generation stage.

6.3 How RI-RAG Addresses the Investigated Problems

- **A. Supporting Traceability:** RI-RAG separates the semantic retrieval entry point from the authoritative database. A retrieved candidate can therefore be resolved to a specific relational record before its content is supplied to subsequent processing. This provides an explicit record-resolution mechanism that can support evidence-grounded generation.

```
# Vector search identifies the record
matched_id = vector_results["ids"][0][0]
```

```
# Retrieve authoritative record
record = self.db.fetch_by_id(matched_id)
```

- **B. Centralizing Metadata Updates:** Metadata and full content remain in the relational source of truth. Updates can therefore be performed at the relational layer without requiring duplication of the complete payload in the vector store.

```
-- Metadata update through SQL
UPDATE file
SET grade = 9.9
WHERE file_id = 'DOC-001';
```

- **C. Supporting Transactional Storage and Access Control:** MySQL provides transactional and privilege mechanisms that can be configured for authoritative records. Their effectiveness depends on database configuration and was not comprehensively evaluated in the present benchmark.

- **D. Supporting Debugging and Content Evolution:** Because authoritative content and metadata are stored relationally, engineers can inspect the corresponding record and its metadata when investigating retrieval or generation behavior. The impact on re-indexing requirements depends on which fields are modified.
- **E. Reduced Vector-Layer Memory Representation:** RI-RAG removes the need for the vector layer to maintain the complete authoritative payload, retaining instead the semantic representation and lightweight identifiers required for retrieval. Although introducing a relational database adds another component with its own memory requirements, relational systems provide configurable mechanisms for controlling resource consumption, including buffer management, connection management, indexing, and query optimization. This allows the additional database overhead to be optimized independently while preserving the reduction in data maintained by the vector layer. In the prototype evaluation, this architectural change reduced the measured vector-store representation by 24.70

7 BROADER IMPACT & TARGET APPLICATIONS

The RI-RAG architecture may be applicable to systems where semantic retrieval and authoritative structured storage have different operational requirements.

- **Enterprise Cybersecurity Operations:** RI-RAG could be used to connect semantic retrieval of security alerts with authoritative incident records, historical evidence, or operational playbooks. Database and application-level access policies could be applied to the authoritative records.
- **Enterprise Knowledge Systems:** Systems containing large authoritative records may use the architecture to maintain semantic retrieval information separately from complete relational payloads. The resulting resource behavior would depend on payload size, record count, database configuration, and workload characteristics.
- **Cloud and Infrastructure Applications:** The architecture could allow some deployments to maintain large authoritative payloads in relational storage rather than duplicating them within the vector layer. The economic implications remain workload- and infrastructure-specific and require additional evaluation.

8 SOURCE CODE AND IMPLEMENTATION

The implementation details, database schema scripts, and synchronization pipelines for the Relational-Indexed Retrieval-Augmented Generation (RI-RAG) architecture are made publicly available to support reproducibility. The complete source code and architectural modules can be accessed on GitHub:

<https://github.com/lmav-jpeg/RI-RAG>

Contributions, issue tracking, and additional benchmarks are welcomed through the repository.

9 CONCLUSION & FUTURE WORK

This work introduced Relational-Indexed Retrieval-Augmented Generation (RI-RAG), a hybrid retrieval architecture that separates semantic indexing from authoritative relational storage. The prototype uses a vector database as the semantic retrieval layer while maintaining MySQL as the authoritative source of truth for complete document content and structured metadata. Retrieved record identifiers are resolved through the relational database before authoritative content is returned.

The experimental results provide an initial characterization of the architecture. In the refined Round 2 benchmark, RI-RAG reduced the measured materialized vector-store representation from 63,954 bytes for Standard RAG to 48,160 bytes, corresponding to a 24.70% reduction under the evaluated 13-record configuration. This measurement concerns the materialized vector-store representation and should not be interpreted as an equivalent reduction in total process or system RAM.

The same benchmark showed an average retrieval latency of 17.75 ms for RI-RAG compared with 7.11 ms for Standard RAG. The additional latency is consistent with the relational hydration step required to resolve the retrieved record identifier into authoritative MySQL content.

Round 3 evaluated retrieval correspondence and functional behavior. RI-RAG correctly retrieved nine of ten ground-truth records at top-1, corresponding to 90.0% top-1 retrieval accuracy, with a precision of 0.9000 and an F1 score of 0.9474. The single mismatch, between DOC-003 and DOC-009, demonstrates that semantic retrieval remains dependent on the quality of the lightweight semantic representations. The evidence-grounding test successfully resolved DOC-001 through the relational layer, while the database-boundary test verified execution under the configured test_user account.

For RI-RAG to perform effectively, there must be an adequate semantic relationship between the lightweight summary stored in the vector database and the original authoritative content stored in the relational database. The summary does not need to reproduce the full content, but it must preserve sufficient semantic information for the retrieval task. AI-based techniques may be used to generate such semantic representations and to evaluate their correspondence with the authoritative content.

The present dataset is intentionally small and synthetic, and the controlled payload expansion does not establish behavior on naturally occurring large enterprise documents. Future experiments should therefore evaluate substantially larger corpora and payloads while measuring retrieval accuracy, vector-store representation size, database storage, latency, throughput, and process-level resource consumption separately. Payload size and record count should be varied independently to determine how the architecture behaves as the authoritative workload grows.

A further objective is to investigate whether the representation-size difference observed in the prototype becomes more pronounced as document payloads increase. Such experiments would provide stronger evidence regarding the central hypothesis of RI-RAG: that semantic retrieval can remain effective while large authoritative payloads are maintained outside the vector layer.

10 AI USAGE ACKNOWLEDGEMENT

This work was gratefully inspired by concepts explored in “Vector Databases Demystified” by Abhinav Verma (Published on Medium). Artificial intelligence tools were utilized in the preparation, code implementation, and generation of this report. Specifically, AI models were prompted and consulted to design and test implementation scripts, evaluate architecture benchmarks, and synthesize and structure the content of this document. AI tools were also used for English language editing, formatting assistance, and LaTeX code structuring. All core concepts, system design, experimental direction, and conclusions remain the original work of the author(s).

REFERENCES

- [1] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. (April 2024). <https://www.microsoft.com/en-us/research/publication/from-local-to-global-a-graph-rag-approach-to-query-focused-summarization/>
- [2] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2025. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. arXiv:2405.14831 [cs.CL] <https://arxiv.org/abs/2405.14831>
- [3] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. 2024. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In *NeurIPS 2025*. <https://www.microsoft.com/en-us/research/publication/retrievalattention-accelerating-long-context-llm-inference-via-vector-retrieval/>